MIT Libraries | DSpace@MIT

MIT Open Access Articles

Clonable key fobs: Analyzing and breaking RKE protocols

The MIT Faculty has made this article openly available. ***Please share*** how this access benefits you. Your story matters.

Citation: Gesteira-Miñarro, R., López, G. & Palacios, R. Clonable key fobs: Analyzing and breaking RKE protocols. Int. J. Inf. Secur. 24, 150 (2025).

As Published: https://doi.org/10.1007/s10207-025-01063-7

Publisher: Springer Berlin Heidelberg

Persistent URL: https://hdl.handle.net/1721.1/159387

Version: Author's final manuscript: final author's manuscript post peer review, without publisher's formatting or copy editing

Terms of Use: Article is made available in accordance with the publisher's policy and may be subject to US copyright law. Please refer to the publisher's site for terms of use.

Massachusetts Institute of Technology

ACCEPTED MANUSCRIPT

Clonable key fobs: Analyzing and breaking RKE protocols

This Accepted Manuscript (AM) is a PDF file of the manuscript accepted for publication after peer review, when applicable, but does not reflect post-acceptance improvements, or any corrections. Use of this AM is subject to the publisher's embargo period and AM terms of use. Under no circumstances may this AM be shared or distributed under a Creative Commons or other form of open access license, nor may it be reformatted or enhanced, whether by the Author or third parties. By using this AM (for example, by accessing or downloading) you agree to abide by Springer Nature's terms of use for AM versions of subscription articles: https://www.springernature.com/gp/open-research/policies/accepted-manuscript-terms

The Version of Record (VOR) of this article, as published and maintained by the publisher, is available online at: https://doi.org/10.1007/s10207-025-01063-7. The VOR is the version of the article after copy-editing and typesetting, and connected to open research data, open protocols, and open code where available. Any supplementary information can be found on the journal website, connected to the VOR.

For research integrity purposes it is best practice to cite the published Version of Record (VOR), where available (for example, see ICMJE's guidelines on overlapping publications). Where users do not have access to the VOR, any citation must clearly indicate that the reference is to an Accepted Manuscript (AM) version.

Accepted manuscript

ACCEPTED MANUSCRIPT

International Journal of Information Security manuscript No.
(will be inserted by the editor)

Clonable key fobs: Analyzing and breaking RKE protocols

Roberto Gesteira-Miñarro[a,1], **Gregorio López**[b,1], **Rafael Palacios**[c,1,2]

[1] Institute for Research in Technology, Comillas Pontifical University, Alberto Aguilera 23, 28015 Madrid, Spain
[2] Cybersecurity at MIT Sloan, Massachusetts Institute of Technology, 77 Massachusetts Avenue, Cambridge, MA 02139, USA

Received: date / Accepted: date

Abstract The automotive industry has been a target for cyber criminals for decades. New regulations have come into force in the automotive industry and manufacturers must take cybersecurity into account. One of the most interesting vehicle systems is the Remote Keyless Entry (RKE) system, which allows users to lock and unlock their cars, among other actions, with a remote control integrated in the car key. If this system is compromised, a malicious user could gain access to a vehicle remaining unnoticed. This paper presents the identification and analysis of a vulnerability in an RKE protocol that can be exploited to gain access to the car at any time, thus cloning the key fob. The reverse-engineering methodology used to uncover the vulnerability is outlined, along with other tested vehicles to show its applicability. A relevant aspect of the research is the fact that only open-source tools and available commercial hardware are needed to perform the analysis. This black-box approach is equally valid to learn RKE protocol features, without the need to extract and analyze ECU firmware, which is considerably more expensive. As a result, a detailed analysis of eight protocols from different manufacturers is shown and they are compared from a cybersecurity point of view, with one of them being totally broken.

1 Introduction

The automotive industry is in continuous growth, driven by innovations in connected vehicle technologies, autonomous systems, and smart infrastructure. Modern vehicles have evolved from traditional mechanical machines into complex digital platforms that integrate a network of sensors, software, and communication systems. These advances improve vehicle performance, human safety, and user experience.

However, new features also introduce new potential cybersecurity vulnerabilities. As vehicles become increasingly interconnected with external networks and personal data, securing automotive systems is crucial to protect both individual users and the broader transportation infrastructure.

For this reason, new regulations (UN Regulation No. 155, 156) have come into force, and car manufacturers must pass cybersecurity certifications before selling their products [38, 39]. This is relevant because the trend is to manufacture increasingly intelligent cars that can be connected to smart infrastructures and improve the quality of driving and people's safety. Moreover, new standards on vehicle-to-everything (V2X) communications are being designed [1], demonstrating that research on the automotive industry is a hot topic.

It is common to see talks about car hacking in cybersecurity conferences such as Black Hat or DEF CON. In fact, every year there is a Car Hacking Village [9] (associated to DEF CON) that joins several researchers on this topic. It is worth mentioning the CyberAuto Challenge and the CyberTruck Challenge [12] as challenge-based models for developing talent and building community among cybersecurity researchers and industry. In addition, there is the Automotive Security Research Group (ASRG) [4]; or VicOne [44], which is a company that focuses on automotive cybersecurity and sponsors the Pwn2Own Automotive competition [45], where hackers from all over the world try to compromise some of the proposed targets for fame and bounties.

In order to improve and test the security of existing and future vehicles, car manufacturers need to rely on research projects, to have an external perspective. This research is not only valuable for vehicle manufacturers, but also for insurance companies, since they need to estimate the insurance rates depending on different factors, for instance, the

[a] rgesteira@comillas.edu
[b] gllopez@comillas.edu
[c] palacios@mit.edu

Accepted manuscript

ACCEPTED MANUSCRIPT

security level of a given vehicle. In fact, there are insurance companies that nowadays are not willing to handle some car models due to their high rate of car theft [40]. Although the entry systems have several mitigation features, news about car theft are very frequent. Cyber criminals always find a way to bypass these protection mechanisms and exploit vulnerabilities. This is the reason why countries such as Canada have recently prohibited the use of hacking devices like Flipper Zero [35], which are known to be used for car hacking.

This paper focuses on one specific subsystem of vehicles, the keyless entry system. There is a lot of research on this topic, namely, there are several types of keyless entry systems, mitigations, attack techniques and analysis approaches. We present an in-depth analysis of many different vehicle models, which is not found in the literature, to the best of our knowledge. However, the main contribution is the explanation of a vulnerability found during the development of our research work. In brief, this vulnerability allows to clone a key fob and thus lock or unlock the doors of the car at any time, remaining unnoticed, with the only requirement of capturing a single signal of the target key fob beforehand.

Furthermore, we present a methodology that is based on reverse engineering and employs commercial and open-source hardware and software, which is less expensive and time-consuming than other approaches documented in the literature while having a similar result. For instance, firmware extraction tools can be notably expensive and hard to adapt to different environments, apart from being invasive on the tested device. In addition, having access to the firmware is not enough, since embedded devices normally use machine architectures that might require an IDA Pro license to get support. In fact, there are manufacturers that prefer using custom machine instructions, which can be extremely difficult to reverse-engineer, as there might be few information available, and extra work must be done to be able to analyze the firmware [37]. In contrast, the methodology proposed in this paper is based on a black-box perspective, which is easier to setup, easier to adapt to different vehicles, non-invasive and more accessible for academic researchers, while still being insightful for the analysis.

In summary, the primary contributions are listed below:

- Analysis of 8 different RKE systems.
- Explanation of a vulnerability found in one of the tested vehicles (Type H).
- Reverse engineering methodology for RKE systems.

The rest of this paper is organized as follows: We describe background about keyless entry systems in Section 2, analyze the state of the art in Section 3, propose our methodology in Section 4 and show the results in Section 5. In Section 6, we discuss the results and propose countermeasures, followed by a conclusion in Section 7.

2 Background

The keyless entry system of a car allows users to lock and unlock their vehicles, among other actions such as opening the trunk or starting up the engine remotely. There are two main types of keyless entry systems: Remote Keyless Entry (RKE) and Passive Keyless Entry and Start (PKES) systems.

The RKE system is simple from the user's viewpoint, since they only need to press a button on their key fob, and the action is performed automatically. From the technical point of view, a digital signal is sent by the key fob and the car analyzes that signal and executes the requested action.

Some modern vehicles often include a Passive Keyless Entry and Start (PKES) system, where the user is no longer required to press a button on the key fob. They just need to approach the vehicle and activate the system with the hand or foot. After a bidirectional communication between the vehicle and the key fob, the doors will be unlocked [28]. PKES systems are also used to start the engine.

In terms of technology, PKES systems are very similar to RKE systems. The main difference is that PKES key fobs include a challenge signal from the vehicle to generate the response. Instead of pressing a button on the key fob, the user performs an action on the car, for example, touching the door handle, which is the "Trigger" signal depicted in Figure 1.

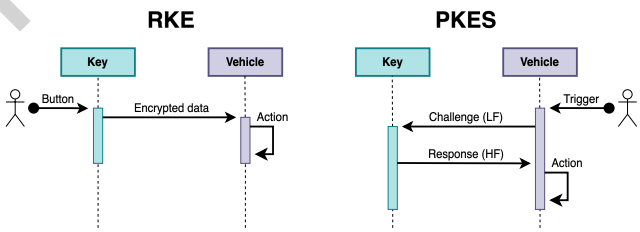

Fig. 1 RKE and PKES message flows.

The communication channel is the air, so it is an insecure channel since anyone with the appropriate tools and knowledge can eavesdrop and tamper with the communication process. RKE systems use non-license frequency bands, such as 433 MHz or 868 MHz in Europe, and 315 MHz in the United States.

In the automotive sector, manufacturers tend to design proprietary cryptographic algorithms to generate short codes that improve the communication security. This fact makes vulnerability assessment much more complex, although still some insights can be obtained with the proposed reverse-engineering methodology. Furthermore, proprietary algorithms are less robust and more likely to be broken with cryptanalysis. Other manufacturers, such as Tesla, use secure cryptography and Bluetooth Low Energy to implement RKE and PKES systems [47].

Accepted manuscript

ACCEPTED MANUSCRIPT

The methodology and tools presented in this paper were developed for evaluating the security level in RKE systems and discovering possible vulnerabilities through reverse engineering. For PKES systems, the level of security depends on the manufacturer implementation, including the protocols and the algorithms used. Reverse-engineering a protocol will not show a clear attack vector in most cases. However, our contribution will help determining and understanding the structure of an RKE protocol. If the protocol is weak, then it is prone to be compromised; otherwise, some insights can be drawn.

3 State of the Art

Most of the research on RKE and PKES systems involves radio frequency attacks or cryptographic attacks. The former attacks try to compromise the logic of a communication protocol, whereas the latter attacks try to break a cipher using several cryptanalysis techniques. Reverse engineering is slightly involved in both scenarios, and there is no clear research on how to successfully reverse-engineer an RKE protocol. To the best of our knowledge, all the projects that show reverse engineering are based on white-box or gray-box approaches like [48] or [18], which is not always possible. Table 1 shows a summary of the contributions of the most relevant papers cited in this section.

RKE systems rely on radio frequency, thus they are vulnerable to replay attacks and jamming. A set of tools to perform these attacks is presented in [25]. Nevertheless, these are *script-kiddie* techniques, since the only requirement is to have the right tools. In [19] a reverse-engineering approach is demonstrated to analyze garage door openers. This research is much more interesting because all the protocol details are discovered, which allows to understand the attacks in a deeper way. A similar resource to learn about reverse engineering in radio frequency protocols can be found in [33] and [31].

3.1 Replay attacks

RKE systems have an intrinsic vulnerability because the vehicle will unlock whenever a valid signal is received. The source of the signal does not matter. Therefore, if a malicious user is able to capture a valid signal, they might be able to unlock the car as well by sending that same signal (replay attack) [19]. Notice that this attack does not depend on the structure of the signal, it is just replayed. Therefore, reverse engineering is not required to perform this attack.

The implementation of rolling codes in RKE signals mitigates replay attacks, as these codes are meant to be used once. The intuition is that the vehicle and the key fob share a pseudorandom number generator (PRNG) algorithm initialized with some seed, so the key fob sends the next step of the algorithm and the vehicle can match the code within a list of valid codes, as shown in Figure 2. As a result, once a code is used, it is marked as invalid to prevent replay attacks.

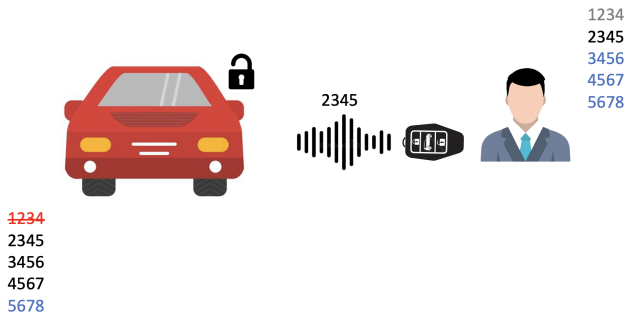

Fig. 2 Operation of rolling codes.

However, there are still ways to perform replay attacks. The limitation is that the vehicle must not receive the code. Hence, if a malicious user captures a valid signal from the key fob and the car is not near, they will be able to replay the signal and the car will behave normally because the signal is valid.

In some cases, a brute-force approach might be useful. There was a talk at DEF CON 32 [16] about a specific type of programmable rolling codes, known as learning codes, that are only 20 bits long. The researcher showed how he was able to reverse-engineer the RKE protocol to find out its structure and perform a 20-bit brute-force attack to unlock these cars.

Regarding mitigations to replay attacks, apart from rolling-code implementations, in [23] they use a timestamp to mitigate replay attacks. Basically, if the car receives an old signal, it will not act in consequence and will discard the code.

3.2 RollJam attack

Furthermore, Samy Kamkar [29] presented in DEF CON 23 a new technique known as RollJam to attack rolling-code implementations in RKE systems. The objective of the RollJam attack is to obtain a set of valid codes that can be used to unlock the doors.

The attack involves using a jamming antenna near the vehicle (i.e. under the vehicle to remain unnoticed) in order to generate interferences in the working frequency band. As a result, the legitimate key fob will not work and the user will press the button several times, as Figure 3 shows.

Meanwhile, the attacker must be near the key fob to capture those signals, which are valid because the car has not received any of them. In the end, the attacker should deactivate

Accepted manuscript

ACCEPTED MANUSCRIPT

Table 1 Summary of contributions of state-of-the-art papers.

Title	Authors	Main contribution
"My other car is your car: compromising the Tesla Model X keyless entry system" [47]	L. Wouters, B. Gierlichs, and B. Preneel	The authors reverse-engineer the keyless entry system of Tesla Model X and find a way to bypass the cryptographic security measures to pair a new key to the car using an unauthorized firmware update
"Fast, furious and insecure: passive keyless entry and start systems in modern supercars" [48]	L. Wouters, E. Marin, T. Ashur, B. Gierlichs, and B. Preneel	The PKES system of high-end cars is reverse-engineered by firmware extraction and analysis. They show a proof-of-concept attack where they can clone a Tesla Model S key fob in a matter of seconds
"Lock It and Still Lose It - on the (in)Security of Automotive Remote Keyless Entry Systems" [18]	F. D. Garcia, D. Oswald, T. Kasper, and P. Pavlidès	The authors show vulnerabilities in the RKE system of the Volskwagen group, being able to exploit them to clone a key fob after a single signal capture. On the other hand, they present a correlation attack against Hitag2 to clone a key fob
"Key is in the Air: Hacking Remote Keyless Entry Systems" [25]	O. Ibrahim, A. Hussain, G. Oligeri, and R. Pietr	Presents an improvement to Samy Kamkar's RollJam attack, where the victim is forced to use the mechanical key to open their car instead of waiting until the jammer is deactivated by the attacker
"Garage Door Openers: A Rolling Code Protocol Case Study" [19]	A. Ghanem and R. Al-Tawy	The authors reverse-engineer three rolling-code protocols for garage doors using Universal Radio Hacker [36]
"Breaking Learning Codes: How I discovered that I can unlock thousands of cars in my country" [16]	D. Erazo	The author analyzes key fob signals and finds out that it uses 20-bit learning codes, allowing to perform a brute-force attack in order to open this type of car
"Drive It Like You Hacked It: New Attacks and Tools to Wirelessly Steal Cars" [29]	S. Kamkar	Replay attacks and countermeasures on RKE systems are presented. The new technique RollJam is shown and proven against cars with rolling-code implementations
"RollBack: A New Time-Agnostic Replay Attack Against the Automotive Remote Keyless Entry Systems" [11]	L. Csikor, H. W. Lim, J. W. Wong, S. Ramesh, R. P. Parameswarath, and M. C. Chan	The authors show how they discovered a flaw in the key fob resynchronization process, being able to send several already-used signals to the vehicle that were captured beforehand in order to resynchronize the key fob and thus unlock the car
"Relay Attacks on Passive Keyless Entry and Start Systems in Modern Cars" [17]	A. Francillon, B. Danev, and S. Capkun	A relay attack is implemented, with and without cables. An in-depth analysis of the attack concept is shown, along with possible countermeasures
"Gone in 360 Seconds: Hijacking with Hitag2" [41]	R. Verdult, F. D. Garcia, and J. Balasch	The authors compromise immobilizers that use Hitag2 by finding vulnerabilities on the algorithm. They are able to run three distinct attacks that succeed in six minutes in order to start the car engine
"One Car, Two Frames: Attacks on Hitag-2 Remote Keyless Entry Systems Revisited" [26]	R. Benadjila, M. Renard, J. Lopes-Esteves, and C. Kasmi	Hitag2 is reviewed and analyzed for RKE systems. The authors show new vulnerabilities that can be exploited in a few minutes with dedicated hardware for a brute-force attack
		Our contribution
"Clonable key fobs: Analyzing and breaking RKE protocols"	R. Gesteira-Miñarro, G. López and R. Palacios	We use a black-box reverse-engineering methodology to analyze RKE protocols and learn their structure. We find and exploit a vulnerability using this methodology. We only use available hardware and software tools to get equivalent results compared to white-box and gray-box approaches

the jamming antenna and use one of the first codes captured to open the car and make the victim think that the key fob is working again [29]. Otherwise, the user might open the car with the physical key, which ensures that captured codes have not being used and improves Samy Kamkar's approach [25].

Regarding jamming techniques, in [6] it was discovered that RKE key fobs employ an envelope detector, which is much vulnerable to pulsed electromagnetic interference than to continuous interference. The researchers show that by using a synchronous detector, the issue is mitigated.

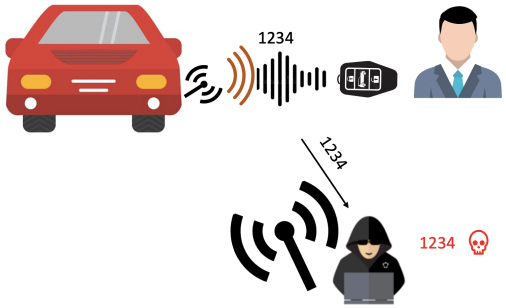

Fig. 3 RollJam attack scenario on an RKE system.

3.3 RollBack attack

RollJam attack is sophisticated and hard to implement. Another vulnerability was found in rolling-code implementations. The attack was named RollBack [11] and it was presented in Black Hat USA 2022.

ACCEPTED MANUSCRIPT

The researchers found that sending several already-used and consecutive signals, the vehicle will resynchronize the key fob and perform the encoded action.

This attack is much more powerful, dangerous and easy to perform. The attacker only needs to capture several codes that the vehicle has received (the type of action does not matter). The researchers show a proof-of-concept video demonstrating that they can reuse this set of consecutive signals multiple times to open the vehicle. Nevertheless, not every vehicle brand is vulnerable to this attack.

3.4 Relay attacks

These attacks only affect PKES systems because of how the protocols work. Recall that once the "Trigger" action is completed, the vehicle sends a cryptographic challenge that only the legitimate key fob configured in the car is supposed to solve. If the response from the key fob is correct, then the vehicle will perform the required action.

The attack scenario requires two malicious attackers, one near the key fob and one next to the car. The attacker next to the car performs the "Trigger" action and relays the challenge signal to the attacker that is next to the key. Then, the challenge signal is amplified so that the key fob responds (as if the signal was sent by the car). Finally, the response signal is relayed to the attacker next to the vehicle and amplified, so that the vehicle unlocks the doors because the response message is correct [3]. Furthermore, attackers can potentially start the engine because the process is mostly the same. Figure 4 illustrates how the attack is performed.

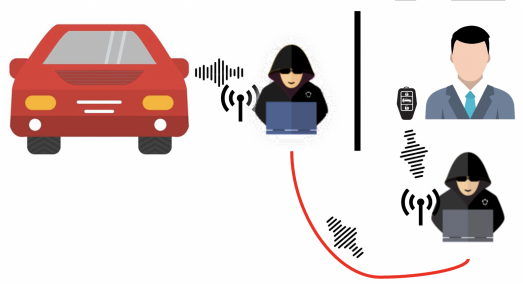

Fig. 4 Relay attack scenario on a PKES system.

As can be seen, relay attacks on PKES systems are dangerous because attackers can potentially steal the car and go away. The devices needed for the attack are not expensive, and the communication between the attackers can be wired or wireless [17].

Regarding mitigations to relay attacks on PKES systems, in [46] the researchers propose the use of Bluetooth Low Energy and several parameters like RSSI, RTT, GPS or Wi-Fi access points to precisely determine whether the key fob is close to the vehicle or not. On the other hand, [28] proposes the use of RF fingerprinting. They basically train a classification model with legitimate signals from different key fobs, so that a malicious signal sent from an SDR device or radio dongle is detected.

3.5 Cryptography

This section covers related work on cryptographic schemes used in the context of RKE/PKES systems and RFID immobilizers. Most of the research is about possible cryptographic attacks, side-channel attacks or implementations on ciphers such as Hitag2 and KeeLoq, which are widely used in the automotive sector.

3.5.1 Hitag2

In [18], researchers do a deep analysis on the Hitag2 cryptographic algorithm (designed by NXP), which is used by some manufacturers for rolling-code RKE systems as well as RFID immobilizer protocols. They successfully found a way to break the Hitag2 cipher with a few samples.

In [41], the researchers found a way to compromise the cipher and unlock a vehicle in around 6 minutes with commercial hardware. Further analyses of the Hitag2 cryptosystem for RFID immobilizers and RKE systems can be found in [7], where an optimized exhaustive search algorithm is developed to forge Hitag2 radio signals from at least two eavesdropped rolling codes.

An OpenCL implementation to break Hitag2 is described in [26]. Compared to FPGA implementations, it is not efficient, but it is less expensive. In the worst case, they successfully break the cipher after 11 hours. An optimized guess-and-determine attack is proposed in [43]. They claim that their implementation is capable of recovering the cryptographic key with 100% success rate and only two RKE signal samples.

3.5.2 KeeLoq

KeeLoq is a block cipher owned by Microchip that was used for rolling-code implementations. After the algorithm was leaked, a lot of research aroused to find vulnerabilities to break the cipher. A curated list of cryptanalysis and side-channel approaches on KeeLoq is presented in [15]. Although there are ways to compromise the cipher, none of them seems to be applicable in RKE systems. Furthermore, KeeLoq has been deprecated, at least in rolling-code implementations.

KeeLoq is based on a Non-Linear Feedback Shift Register (NLFSR) scheme, has a 64-bit key and operates on 32-bit

ACCEPTED MANUSCRIPT

blocks [8]. Research has shown that KeeLoq can be compromised with slide attacks and meet-in-the-middle attacks, although the implementations require a considerable amount of plaintexts and expensive hardware. [27] presents a practical key recovery attack against KeeLoq that requires 2^{16} known plaintexts. It is based on the slide attack and a novel meet-in-the-middle attack. The fully implemented attack requires 65 minutes to obtain the required data and 7.8 days of calculations on 64 CPU cores. A variant attack needs only 3.4 days on 64 CPU cores.

KeeLoq is also vulnerable to side-channel attacks. [14] presents differential power analysis attacks in numerous commercial products that use KeeLoq as a rolling code implementation. They successfully reveal the secret key of a remote transmitter and the manufacturer key stored in a receiver. As a result, a remote control can be cloned with only ten power traces, resulting in a practical key recovery in a few minutes. Once the manufacturer key is known, they demonstrate how to disclose the secret key of a remote control just by eavesdropping at most two messages from a remote key fob. Other researchers show that physical side-channel attacks can be used to recover the 64-bit KeeLoq key stored in a key fob [13] or the 64-bit manufacturer key stored in the receiver [30].

3.5.3 Other ciphers

The researchers in [18] analyze the security of four Volkswagen group vehicles. The approach is gray-box-based since they were allowed to extract the firmware from the corresponding electronic control units (ECUs) to look for cryptographic objects and identify the cryptographic scheme. VW-1 vehicles are based on obscurity and a Linear Feedback Shift Register (LFSR), whereas the rest of the models use an 8-byte block cipher algorithm. VW-2 and VW-3 use a cipher known as AUT64, and VW-4 uses XTEA (eXtended Tiny Encryption Algorithm); both are Feistel block ciphers. The critical problem found is that all VW vehicles used the same global key for encryption. Therefore, once the key is found, any malicious user could clone a remote key fob to open any VW vehicle after eavesdropping a few signals. Despite being an important cybersecurity research, Volkswagen group did not allow the researchers to disclose all the information, methodology and techniques employed during the process. In [24], these researchers continue to analyze the AUT64 block cipher, which is proprietary and closed-source. Again, they successfully identified some weaknesses that could compromise the algorithm.

On the other hand, [42] focuses on the Megamos Crypto cipher, used by a wide variety of vehicle manufacturers and based on Galois LFSR and NLFSR.

4 Methodology

This section shows how open-source and available tools are used in our work. Moreover, the reverse-engineering methodology is defined in order to obtain the necessary data to analyze different RKE protocols.

4.1 Radio frequency tools

This section describes a set of tools that are useful for working with radio frequency signals, both hardware devices and software programs.

4.1.1 Hardware

HackRF One, from Great Scott Gadgets, is a half-duplex transceiver that can operate at frequencies from 1 MHz to 6 GHz and can reach a rate of 20 million samples per second. In addition, it is an open-source hardware device [21]. With HackRF One and the appropriate software, the real-time frequency spectrum can be visualized. Moreover, it allows to capture signals and replay them. HackRF One is perfect for performing replay attacks.

There is another device from Great Scott Gadgets, known as YARD Stick One, that accepts different types of digital modulations (ASK/OOK, GFSK, 2FSK, 4FSK, MSK) and binary rates of up to 500 kbps [22]. This gadget can be used to capture and decode signals, and to generate synthetic signals from binary data. Therefore, YARD Stick One can be used to perform replay attacks from digital signals instead of analog signals.

Regarding hardware costs, HackRF One can be purchased for around €300, and YARD Stick One for around €150. Great Scott Gadgets does not provide a single cost for their products, but rather propose several online shops where they can be purchased from different countries, so the cost may vary depending on the website.

4.1.2 Software

To interact with a HackRF One gadget, GNU Radio Companion is a good option. It is an open-source project that provides a graphical programming environment based on signal processing blocks for interacting with SDR devices [20], such as HackRF One.

GQRX [34] is a program based on GNU Radio Companion that displays the frequency spectrum in a waterfall model and is able to process the received radio signals. It can be used to identify the working frequency of a given remote key fob with HackRF One.

On the other hand, YARD Stick One can only be used with `rfcat` [5], which is a Python library.

Accepted manuscript

ACCEPTED MANUSCRIPT

The program `inspectrum` displays the power of a signal in time and frequency. It is used to analyze signal captures and extract their characteristics and even the symbols encoded as bits [32].

4.2 Signal captures

HackRF One will be used to perform a basic reconnaissance of radio frequency and to capture key fob signals with GNU Radio Companion. Figure 5 shows the process of capturing a signal and saving it to a file.

4.3 Signal analysis

The captured signals will be processed by `inspectrum` and the power of the signal will be plotted in time and frequency axes. RKE signals are usually modulated using digital schemes such as Amplitude Shift Keying, particularly On-Off Keying (ASK/OOK); or Frequency Shift Keying, especially with two carrier frequencies (2FSK). Figure 6 and Figure 7 show both kinds of modulations, respectively.

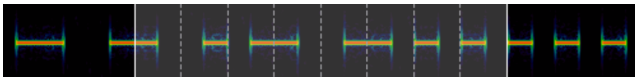

Fig. 6 ASK/OOK-modulated signal.

Fig. 7 2FSK-modulated signal.

The signals carry modulated symbols, which can be extracted with `inspectrum`. It is common that the information bits are Manchester-encoded. In practice, this means that a bit `0` will be encoded as `01` and a bit `1` will be encoded as `10` (or the other way around, depending on the convention). Manchester encoding allows the receiver to keep synchronization at symbol level because there will not be a long period of continued signal power. Even a burst of `0`s or `1`s will result in an alternating sequence of `0`s and `1`s. As a result, `inspectrum`'s plot will show signal pulses with two different widths, so it is easy to identify that the signal is Manchester-encoded.

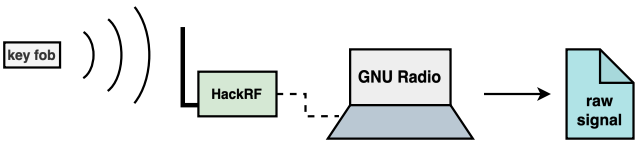

Fig. 5 Signal capture procedure.

Furthermore, RKE signals usually start with a synchronization sequence. This is just a burst of `1`s. Figure 8 shows an example of a Manchester-encoded 2FSK-modulated synchronization sequence. The last bit of the synchronization sequence is `0`, which indicates the start of the information frame.

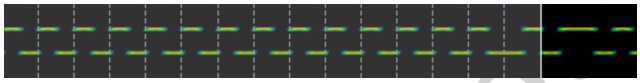

Fig. 8 Synchronization sequence.

Once a signal binary data is extracted, we can take several samples and look for coincidences and differences between distinct frames of the same RKE system. It is possible to guess several fields and extract features of the protocol. The overall process is depicted in Figure 9.

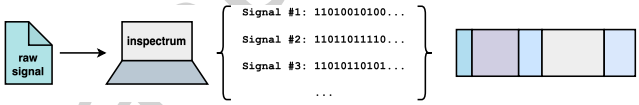

Fig. 9 Signal analysis procedure to find protocol structure.

5 Results

In this section, several examples of RKE signals are shown and analyzed to guess the protocol structure. The relevant part of the encoded signals will be shown in tables, differentiating between commands ("Open" and "Close" and sometimes "Trunk" or "Lights"). Synchronization sequences will not appear in the tables. Each signal has a number to keep an ordering (i.e. "Open 1" was captured before "Open 2"). Furthermore, the names of the tested models are not disclosed for privacy reasons.

5.1 Type A

This RKE key fob uses 2FSK modulation and encodes 168 bits with Manchester encoding. This is an example: `ffffffffffffe7b5fee66491d0592c722769755eb`. There is a synchronization sequence of 6 bytes `ff` ending with a byte `fe`. Therefore, the information bits are `7b5fee66491d0592c722769755eb`.

The codes of Table 2 show some similarities. Code pieces that do not change between signals are highlighted in bold, whereas pieces that change between commands are underlined. As a result, the structure of the frame can be guessed. Figure 10 shows the information structure in bits:

Accepted manuscript

ACCEPTED MANUSCRIPT

Table 2 Information bits for Type A.

Command	Code (hexadecimal)
Open 1	7beb**ee66491d**f2182cd18b4b59d4
Open 2	7ba3**ee66491d**c9fba0de9104d816
Open 3	7bfd**ee66491d**ca21ea6edadd9fc5
Open 4	7b0d**ee66491d**a5f7549b212f4660
Open 5*	7b3f**edf0c83c**cf4629f8133d4e07
Close 1	3b59**ee66491d**0585e74729e8e34c
Close 2	3b19**ee66491d**f15688a309201984
Close 3	3b59**ee66491d**0585e74729e8e34c
Close 4	3b6c**ee66491d**1f4902422d6b8f22
Close 5*	3b37**edf0c83c**c6b518a8185aae51
Trunk 1*	5d97**edf0c83c**b14d4a5ba82bee3f

- The first byte indicates the command (`7b` for "Open", `3b` for "Close" and `5d` for "Trunk").
- The second byte seems to be random, but it can also be a checksum for error detection.
- The next 4 bytes hold a key identifier. Signals marked with * have a different identifier. This happens because these signals were captured from another RKE key of the same car manufacturer.
- The last 8 bytes are random.

Fig. 10 Protocol structure for Type A.

5.2 Type B

These signals are composed by two Manchester-encoded frames of 557 bits and 101 bits, using ASK/OOK modulation. This is an example of this RKE signal: `1d1d1d

ACCEPTED MANUSCRIPT

In fact, Type C is Volkswagen. The above frame structure is presumably correct because it is the same as the one disclosed in [18] (either VW-2, VW-3 or VW-4). Although the 8-byte field is not random but encrypted with AUT64 or XTEA, from a black-box approach it can only be stated that this field looks random or possibly encrypted. Figure 11 shows the structure of the protocol.

Fig. 11 Protocol structure for Type C.

5.4 Type D

This key fob uses 2FSK modulation with Manchester encoding. One signal is composed of 4 frames. The first and third frames are just synchronization sequences, and the second and fourth contain almost the same information. Each information frame is composed of 172 bits and starts with 44 bits `1` (`fffffffffff` in hexadecimal). This is an example of such information frames (removing the initial 44 bits): firstly `93f1bdf759aa55953107544a0c69698f` and secondly `93f1bdf659aa55953107544a0c69691e`. Both signals are exactly the same except for a nibble that always changes from `7` to `6` and the last byte, which looks random.

Fig. 12 Protocol structure for Type D.

After taking several samples, some coincidences were found. According to Table 5, this might be the structure of the information bits, depicted in Figure 12:

- The first 4 bytes are always the same except for the last nibble, which is `7` for the first frame and `6` for the second.
- The following 3 bytes are always the same. This could be a key identifier.
- Then there are 8 bytes that look random between different samples.
- Finally, there is a byte that seems to be random, but it can also be a checksum.

In fact, the last byte was really interesting, because between the two information frames of a given signal, this byte was the only one that was not predictable at first glance.

Table 5 Information bits for Type D.

Command	Code (hexadecimal)
Open 1	93f1bdf759aa55953107544a0c69698f 93f1bdf659aa55953106544a0c69691e
Open 2	93f1bdf759aa55b33e8293dce810aca2 93f1bdf659aa55b33e8293dce810ac33
Open 3	93f1bdf759aa55b250082f0c69e02913 93f1bdf659aa55b250082f0c69e02982
Open 4	93f1bdf759aa55edd8d1ae5697d527bd 93f1bdf659aa55edd8d1ae5697d5272c
Close 1	93f1bdf759aa5591feccdf4419205544 93f1bdf659aa5591feccdf44192055d5
Close 2	93f1bdf759aa55eba94c54636b8ca70f 93f1bdf659aa55eba94c54636b8ca79e
Close 3	93f1bdf759aa5525a98a4eedc41679b2 93f1bdf659aa5525a98a4eedc4167923
Close 4	93f1bdf759aa55f49c2593deeb3f7918 93f1bdf659aa55f49c2593deeb3f7989
Trunk 1	93f1bdf759aa55f96208816dbbfaf9f5 93f1bdf659aa55f96208816dbbfaf964
Trunk 2	93f1bdf759aa55df9aaf96090fea1cde 93f1bdf659aa55df9aaf96090fea1c4f
Trunk 3	93f1bdf759aa55548ed1268fac85b7e9 93f1bdf659aa55548ed1268fac85b778
Trunk 4	93f1bdf759aa55e1953c627b8c7e6549 93f1bdf659aa55e1953c627b8c7e65d8

Using a tool called RevEng [10], it was discovered that the last byte was, in fact, a Cyclic Redundancy Check (CRC) code of the previous part. The parameters of the CRC code could be extracted as follows:

```
# ./reveng -w 8 -s \
> 93f1bdf759aa55953107544a0c69698f \
> 93f1bdf659aa55953107544a0c69691e \
> 93f1bdf759aa55b33e8293dce810aca2 \
> 93f1bdf659aa55b33e8293dce810ac33 \
>
width=8  poly=0x2f  init=0xe0  refin=false
refout=false  xorout=0x00  check=0x56
residue=0x00  name=(none)
```

As can be seen, using only 4 codes (that is, 2 signals), the tool is able to obtain the CRC parameters. To verify that the result is correct, we can generate the CRC values of the following codes and see that they coincide with Table 5. Although this is a great outcome, it is not useful because the codes have an 8-byte field that looks random and does not seem to be predictable. Actually, it is likely to be encrypted because there is no clear command data, similar to Type B.

```
# ./reveng -c -w 8 -p 2f -i e0 \
> 93f1bdf759aa55b250082f0c69e029 \
> 93f1bdf659aa55b250082f0c69e029 \
> 93f1bdf759aa55edd8d1ae5697d527 \
> 93f1bdf659aa55edd8d1ae5697d527 \
>
13
82
bd
2c
```

Accepted manuscript

ACCEPTED MANUSCRIPT

5.5 Type E

These signals are modulated in ASK/OOK, and there are a total of 328 bits (encoded in Manchester) divided into three frames ($132 + 132 + 64$). The three frames hold the same binary data, although the last frame is truncated. Curiously, there is no synchronization sequence at the beginning.

This is a full signal to unlock the car:
`cfc4a7df931fe4af33f129f7e4c7f92bc`
`cfc4a7df931fe4af33f129f7e4c7f92bc`
`cfc4a7df931fe4af`

As can be seen, the relevant information appears in one of the 132-bit frames, repeated for redundancy: `cfc4a7df931fe4af33f129f7e4c7f92bc`. Table 6 shows some samples for the same type of vehicle.

Table 6 Information bits for Type E.

Command	Code (hexadecimal)
Open 1	**cf**c4a7df931f**e4**af**33**f129f7e4c7f**9**2bc
Open 2	**cf**482fab7998**e4**e6**33**d20beade663**9**398
Open 3	**cf**07599391cc**e4**03**33**c1d664e4733**9**00c
Open 4	**cf**242fa179ab**e4**cb**33**c90be85e6af**9**32c
Close 1	**cf**5bac7f7e9d**e4**5433d6eb1fdfa77**9**150
Close 2	**cf**03622c13d2**e4**29**33**c0d88b04f4b**9**0a4
Close 3	**cf**0e14c3e3c4**e4**3f**33**c38530f8f13**9**0fc
Trunk 1	**cf**8619c1a932**e4**ee**33**e186706a4cb**9**3b8

Some bits are always the same in all samples (highlighted in bold). The underlined nibbles have two bits with the same value (namely `c-d-e-f`, `3-7-b-f` and `0-4-8-c`). The hexadecimal representation is not helpful for this RKE protocol.

These codes must be encrypted or encoded since there is no clear sign that indicates the action ("Open", "Close" and "Trunk"), as happened with Type B as well. Nevertheless, these codes look more obfuscated, and the coincidences do not make sense. Therefore, it is hard to guess the structure of the frame or what those fixed bits are used for.

5.6 Type F

This RKE key fob uses ASK/OOK modulation and encodes 312 bits with Manchester encoding. This is an example: `ffffffffffffffffffffffffffffffffffffffff`
`ffffffffffffffffffffffffffffffffffffffff`
`ffffffffffffffffffffffffffffffffffffffff`
`

ACCEPTED MANUSCRIPT

ploys a complex protocol which is very difficult to reverse-engineer.

Two different models of this manufacturer were analyzed (high-end and low-end cars). After analyzing some samples, these conclusions are drawn:

- A long synchronization sequence is sent.
- After a certain period of time there is a set of 15-byte separated frames. There are 3 codes in the first set for the low end car and 4 codes in the high end car.
- After some time, another set of frames is sent, which hold 9.5-byte codes and have a longer time separation. There can be from 2 to 9 codes in this second set.
- Within a set, the codes follow a decreasing sequence with an offset that has a small variation. It is almost a linear decreasing sequence.
- In the first set, there are 7.5 constant bytes and then a hexadecimal digit that holds the command to be executed (`7` for "Open", `b` for "Close", `d` for "Trunk" and `e` for "Lights"). This result makes sense because:
  - `7 = "0111"`
  - `b = "1011"`
  - `d = "1101"`
  - `e = "1110"`

Table 8 shows some samples of this type of RKE signals. The synchronization sequence is omitted, and the two frame sets are separated by a blank line.

The frames of the first set look very similar among each other. They differ only in two bytes. The last byte seems to be unrelated, so removing the last byte and subtracting two consecutive codes result in a constant difference of `0x40000000000`. As in other RKE protocols, this situation is likely to have a CRC code on the last byte. Indeed, RevEng [10] discovers a CRC setup for the last byte.

Curiously, the codes from the second sets were already used in other samples. Actually, the hexadecimal format for the second sets is not really helpful. These codes should be shifted 2 bits. For example, for "Open 1":

```
>>> hex(0x3fffb56007d3e57dfa8c << 2)
'0xfffed5801f4f95f7ea30'
>>> hex(0x3fffb56007d3e57df980 << 2)
'0xfffed5801f4f95f7e600'
>>> hex(0x3fffb56007d3e57df884 << 2)
'0xfffed5801f4f95f7e210'
>>> hex(0x3fffb56007d3e57df7b8 << 2)
'0xfffed5801f4f95f7dee0'
>>> hex(0x3fffb56007d3e57df6bc << 2)
'0xfffed5801f4f95f7daf0'
>>> hex(0x3fffb56007d3e57df5b0 << 2)
'0xfffed5801f4f95f7d6c0'
>>> hex(0x3fffb56007d3e57df4b4 << 2)
'0xfffed5801f4f95f7d2d0'
>>> hex(0x3fffb56007d3e57df3a8 << 2)
'0xfffed5801f4f95f7cea0'
>>> hex(0x3fffb4e007d3e57ff3a6 << 2)
'0xfffed3801f4f95ffce98'
```

Table 8 Information bits for Type G.

Command	Code (hexadecimal)
Open 1	**fffed9801f4f95f**7f8676fb2ab235e **fffed9801f4f95f**7f4676fb2ab2352 **fffed9801f4f95f**7f0676fb2ab2356 **fffed9801f4f95f**7ec676fb2ab234a 3fffb56007d3e57dfa8c 3fffb56007d3e57df980 3fffb56007d3e57df884 3fffb56007d3e57df7b8 3fffb56007d3e57df6bc 3fffb56007d3e57df5b0 3fffb56007d3e57df4b4 3fffb56007d3e57df3a8 3fffb4e007d3e57ff3a6
Open 2	**fffed9801f4f95f**7f8664e13315630 **fffed9801f4f95f**7f4664e1331563c **fffed9801f4f95f**7f0664e13315638 **fffed9801f4f95f**7ec664e13315624 3fffb56007d3e57dfa8c 3fffb4e007d3e57ffa82
Open 3	**fffed9801f4f95f**7f8656ebe537cf6 **fffed9801f4f95f**7f4656ebe537cfa **fffed9801f4f95f**7f0656ebe537cfe **fffed9801f4f95f**7ec656ebe537ce2 3fffb56007d3e57dfa8c 3fffb4e007d3e57ffa82
Close 1	**fffed9801f4f95f**bf8636c7b18dfd3 **fffed9801f4f95f**bf4636c7b18dfdf **fffed9801f4f95f**bf0636c7b18dfdb **fffed9801f4f95f**bec636c7b18dfc7 3fffb56007d3e57efa80 3fffb4e007d3e57ffa82
Trunk 1	**fffed9801f4f95f**df861453a03fc87 **fffed9801f4f95f**df461453a03fc8b **fffed9801f4f95f**df061453a03fc8f **fffed9801f4f95f**dec61453a03fc93 3fffb56007d3e57f7a86 3fffb4e007d3e57ffa82
Lights 1	**fffed9801f4f95f**ef862665b3aa3a3 **fffed9801f4f95f**ef462665b3aa3af **fffed9801f4f95f**ef062665b3aa3ab **fffed9801f4f95f**eec62665b3aa3b7 3fffb56007d3e57fba85 3fffb56007d3e57fb989 3fffb4e007d3e57ff98e

As can be seen, the above shifts are similar to the signals from the first set of "Open 1". Except for the last result, all of them start with `fffed5801f4f95f7`, which is also similar to the prefix of all the signals from the first sets. The remaining 2 bytes are not a CRC code according to RevEng [10].

The only part that might be changing randomly is a 5-byte chunk (underlined in Table 8), although it looks deterministic. Indeed, the sequence is decreasing, but the differences are not predictable:

ACCEPTED MANUSCRIPT


```
>>> hex(0x676fb2ab23 - 0x664e133156)
'0x1219f79cd'
>>> hex(0x664e133156 - 0x656ebe537c)
'0x0df54ddda'
>>> hex(0x656ebe537c - 0x644f7a71f6)
'0x11f43e186'
>>> hex(0x644f7a71f6 - 0x636c7b18df)
'0x0e2ff5917'
>>> hex(0x636c7b18df - 0x62665b3aa3)
'0x1061fde3c'
>>> hex(0x62665b3aa3 - 0x61453a03fc)
'0x1212136a7'
```

The previous calculations were performed because the protocol seems to be related to timing. As an initial idea, we thought that the key fob had some timer and on each press, the current timestamp was sent as rolling code.

The reason behind this impression was a failed replay attack. We tried to perform a replay attack with HackRF One by capturing a signal from the key fob (far away from the car) and then replay the signal next to the car. On the low-end car, the attack did not work, but on the high-end car it was successful. The only difference in these tests was time: in the first attempt, the time elapsed between capture and replay was more than 10 minutes, whereas in the second attempt the whole attack was finished in less than 2 minutes.

Still much more effort needs to be put for this RKE protocol to fully reverse-engineer it. The fact that there are not many random bytes is promising, but the protocol is still obscure and complex to analyze from a black-box perspective.

5.8 Type H

This type of vehicle was analyzed the same way as the rest, and we detected that all the protocol fields are deterministic and predictable. The frame starts with a 40-byte synchronization sequence, and after that, the information is encoded in 8 bytes. Table 9 shows some examples.

Table 9 Information bits for Type H.

Command	Code (hexadecimal)
Open 1	b3bbe5**b76c2f**5d2c
Open 2	b3bbe4**b76c2f**5d8e
Open 3	b3bbe3**b76c2f**5d61
...	
Open 20	b3bbd2**b76c2f**5d55
Close 1	b3bbd1**b76c2f**5e4d
Close 2	b3bbd0**b76c2f**5eef
Close 3	b3bbcf**b76c2f**5e4b
...	
Close 10	b3bbc8**b76c2f**5ea4
Trunk 1	b3bbc7**b76c2f**57a1
Trunk 2	b3bbc6**b76c2f**5703
Trunk 3	b3bbc5**b76c2f**579a
...	
Trunk 10	b3bbbe**b76c2f**571b

As can be seen, the protocol structure is weak and lacks randomness and encryption. In other words, there is no unpredictable field, everything is deterministic and predictable. Figure 14 shows the structure of the guessed protocol:

- `counter` is a 3-byte field that represents a decreasing counter, one by one.
- `key ID` is a fixed 3-byte number for a given key fob.
- `cmd` is 1 byte that encodes the action to execute in the car (lock, unlock, trunk).
- `crc` is an 8-bit CRC code to detect reception errors.

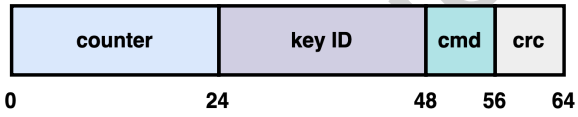

Fig. 14 Protocol structure for Type H.

Before finding that the last byte of the frame was an 8-bit CRC code (like in the previous examples), a valid approach was to use 1-byte brute force, which is affordable. However, since the last byte can be computed as a function of the rest of the frame bytes, the brute-force approach is not actually needed.

As a result, if an attacker were to compromise this vehicle, they only need a single signal capture before performing the attack. This would be the attack strategy:

1. Capture one signal.
2. Extract key fob ID and current counter.
3. Choose the command and decrease counter.
4. Compute the CRC code.
5. Send the crafted signal.

After that, the signal will be valid and accepted by this vulnerable Type H vehicle.

We implemented the attack in a Python script using YARD Stick One and `rfcat`, available in GitHub [2]. It was possible to clone the key fob because we were able to send any key fob command with our tool.

We started a responsible vulnerability disclosure process with the manufacturer months before submitting this paper. They recognized the vulnerability and allowed us to publish our findings.

6 Discussion

Table 10 shows a summary of the previous analysis. For each type of vehicle analyzed, the modulation, number of information bits, number of random bits and the CRC parameters are listed.

ACCEPTED MANUSCRIPT

The "Info. bits" field omits repetitions and synchronization sequences. Thus, it holds the bit length of the protocol frame. Analogously, "Rand. bits" shows the number of bits that are likely to be random or encrypted (in other words, not predictable).

All tested vehicles use ASK/OOK or 2FSK modulations, which means that only simple digital modulations are used to minimize the cost of the emitter and receiver modules on the key fob and the ECU, respectively.

It is interesting that "Rand. bits" field is 64 bits for 5 out of 8 models analyzed. This result is reasonable because of how block ciphers and PRNGs work. According to the number of unpredictable bits, the security of 5 out of 8 models is similar.

Table 10 Summary of information bits.

Type	Modulation	Info. bits	Rand. bits	CRC
A	2FSK	112	64	-
B	ASK/OOK	64	64	-
C	ASK/OOK	96	64	-
D	2FSK	128	64	`p:2f; i:e0`
E	ASK/OOK	132	-	-
F	ASK/OOK	96	64	`p:01; i:00`
G	2FSK	120	40	`p:01; i:fe`
H	2FSK	64	0	`p:7f; i:7a`

Notice how the protocol for Type B is completely random. This means that this manufacturer is using an encryption mechanism to grant security and privacy to the protocol. There is no way for an unauthorized eavesdropper to determine any protocol field (key ID, command, etc.) because they do not have the encryption key. Attackers must identify and cryptanalyze the cipher to break this protocol.

From an offensive point of view, none of the protocols (except Type H) is straightforward to compromise since the block cipher algorithms or PRNGs are proprietary and unknown. We could say that Type G has a lower security level compared to the rest because it relies on a 40-bit value.

Nevertheless, using closed-source algorithms is security by obscurity, which is not recommended because these non-standard implementations usually contain weaknesses. Moreover, these algorithms tend to be leaked in underground forums, as happened with KeeLoq [15]. Indeed, modern cryptography does not rely on private algorithms, but on private encryption keys.

Radio frequency attacks such as replay and relay attacks still work because of the way RKE communication is done. With this reverse-engineering methodology, we are able to learn how the information is structured in the messages sent from the key fob to the car, so we can determine if there are vulnerabilities and the overall level of security of the car.

As a best solution, it would be more secure to use a standard block cipher such as AES, which is known to be very robust, although it would increase the hardware complexity of the key fob. However, if all vehicles used an AES cipher to encrypt their protocol frames, every captured signal would be indistinguishable from other models.

The design of vehicular subsystems, in particular keyless entry systems, is important for the lifecycle of the vehicle. These subsystems must have a long lifetime because repairs and replacements are costly and time-consuming. In addition, a problem with any subsystem of a vehicle would decrease the manufacturer's reputation.

On the other hand, RKE security is very focused on car theft and unauthorized access, but it is also possible to come with a denial of service. In other words, an attacker can use replay attacks to lock a victim out of their vehicle, either by triggering some security protection on the car or either desynchronizing the victim's key fob. In this situation, the manufacturer would likely be blamed, and the victim would rarely believe they were targeted by an attacker.

7 Conclusion

In summary, a reverse-engineering methodology to analyze RKE protocols has been presented. The main contribution is how we found a vulnerability in one of the tested protocols (Type H). It was exploited to predict next rolling codes with a single capture (in other words, clone a key fob). Once the protocol was fully reverse-engineered, YARD Stick One was used to generate and emit future signals, implemented in a Python script. In addition, we present other results using our reverse-engineering methodology.

With this methodology, it is possible to quickly audit RKE protocols and determine if they are vulnerable or not, using a black-box approach, and commercial hardware and open-source software. Although having the knowledge and tools to extract firmware would make the reverse-engineering methodology more precise, it would also make the cost of the project much more expensive.

As a proof of the effectiveness of the proposed method, the results obtained for Type C (Volkswagen) protocol with reverse engineering were correct according to what other researchers did by analyzing the firmware of the ECU [18]. Nevertheless, the black-box approach has some limitations. For instance, it is difficult to distinguish between random bytes and encrypted bytes. Plus, since manufacturers use proprietary ciphers, it is almost impossible to guess the algorithm from this perspective.

In order to improve RKE protocols, manufacturers need to implement a robust algorithm to encrypt the key fob messages. This way, none of the RKE protocols could be identified by the frame structure, and thus, the security and privacy would be preserved.

The automotive industry is in continuous growth and reverse-engineering methodologies need to be adopted by

Accepted manuscript

ACCEPTED MANUSCRIPT

security researchers in order to evaluate the robustness of automotive protocols. This happens because vehicle manufacturers are not willing to publish or disclose any of their designs, which makes security analysis and vulnerability research much more challenging and difficult. On the other hand, new regulations have come into force and car manufacturers must pass a cybersecurity certification before selling their products. With the rise of connected vehicles, shared vehicles and Vehicle-to-Everything communications (V2X) [49], an increase in research projects in this field is expected.

References

1. C-V2X explained. URL https://5gaa.org/c-v2x-explained
2. 7Rocky: `rf_exploit`. URL https://github.com/7Rocky/rf_exploit
3. Alrabady, A., Mahmud, S.: Some attacks against vehicles' passive entry security systems and their solutions. IEEE Transactions on Vehicular Technology **52**(2), 431–439 (2003). DOI 10.1109/TVT.2003.808759
4. Enabling positive Security solutions for the Automotive Market. URL https://asrg.io
5. rfcat (2024). URL https://github.com/atlas0fd00m/rfcat
6. van de Beek, S., Leferink, F.: Vulnerability of Remote Keyless-Entry Systems Against Pulsed Electromagnetic Interference and Possible Improvements. IEEE Transactions on Electromagnetic Compatibility **58**(4), 1259–1265 (2016). DOI 10.1109/TEMC.2016.2570303
7. Benadjila, R., Renard, M., Lopes-Esteves, J., Kasmi, C.: One Car, Two Frames: Attacks on Hitag-2 Remote Keyless Entry Systems Revisited. In: Proceedings of the 11th USENIX Conference on Offensive Technologies, WOOT'17. USENIX Association, USA (2017)
8. Bogdanov, A.: Cryptanalysis of the KeeLoq block cipher. IACR Cryptol. ePrint Arch. **2007** (2007)
9. Car Hacking Village. URL https://www.carhackingvillage.com
10. CRC RevEng: arbitrary-precision CRC calculator and algorithm finder (2024). URL https://reveng.sourceforge.io
11. Csikor, L., Lim, H.W., Wong, J.W., Ramesh, S., Parameswarath, R.P., Chan, M.C.: RollBack: A New Time-Agnostic Replay Attack Against the Automotive Remote Keyless Entry Systems. ACM Transactions on Cyber-Physical Systems **8**(1), 1–25 (2024). DOI 10.1145/3627827
12. Daily, J., Span, M.T.: A Model for Cybersecurity Education through Challenge Events. INCOSE International Symposium **34**(1), 944–957 (2024). DOI 10.1002/iis2.13187
13. Eisenbarth, T., Kasper, T., Moradi, A., Paar, C., Salmasizadeh, M., Manzuri, M.: On the Power of Power Analysis in the Real World: A Complete Break of the KeeLoq Code Hopping Scheme. In: Advances in Cryptology – CRYPTO 2008, pp. 203–220. Springer Berlin Heidelberg, Berlin, Heidelberg (2008). DOI 10.1007/978-3-540-85174-5_12
14. Eisenbarth, T., Kasper, T., Moradi, A., Paar, C., Salmasizadeh, M., Manzuri, M.: Physical Cryptanalysis of KeeLoq Code Hopping Applications. IACR Cryptology ePrint Archive **2008** (2008)
15. Enderlein, R.R.: KeeLoq. Tech. rep., École Polytechnique Fédérale de Lausanne (2010). URL http://www.e7n.ch/data/e10.pdf
16. Erazo, D.: Breaking Learning Codes (2024). URL https://media.defcon.org/DEF%20CON%2032/DEF%20CON%2032%20villages/DEF%20CON%2032%20-%20Car%20Hacking%20Village%20-%20Danilo%20Erazo%20-%20How%20I%20discovered%20and%20hacked%20Learning%20Codes%20of%20the%20key%20job%20of%20a%20car%20assembled%20in%20my%20country.pdf
17. Francillon, A., Danev, B., Capkun, S.: Relay Attacks on Passive Keyless Entry and Start Systems in Modern Cars. IACR Cryptol. ePrint Arch. (2011). DOI 10.3929/ETHZ-A-006708714
18. Garcia, F.D., Oswald, D., Kasper, T., Pavlidès, P.: Lock It and Still Lose It - on the (in)Security of Automotive Remote Keyless Entry Systems. In: Proceedings of the 25th USENIX Conference on Security Symposium, SEC'16, p. 929–944. USENIX Association, USA (2016)
19. Ghanem, A., AlTawy, R.: Garage Door Openers: A Rolling Code Protocol Case Study. In: 2022 19th Annual International Conference on Privacy, Security & Trust (PST), p. 1–6 (2022). DOI 10.1109/PST55820.2022.9851991
20. About GNU Radio (2024). URL https://www.gnuradio.org/about/
21. HackRF One (2024). URL https://greatscottgadgets.com/hackrf/one/
22. YARD Stick One (2024). URL https://greatscottgadgets.com/yardstickone/
23. Greene, K., Rodgers, D., Dykhuizen, H., McNeil, K., Niyaz, Q., Shamaileh, K.A.: Timestamp-based Defense Mechanism Against Replay Attack in Remote Keyless Entry Systems. In: 2020 IEEE International Conference on Consumer Electronics (ICCE), p. 1–4 (2020). DOI 10.1109/ICCE46568.2020.9043039
24. Hicks, C., Garcia, F.D., Oswald, D.: Dismantling the AUT64 Automotive Cipher. IACR Transactions on Cryptographic Hardware and Embedded Systems p. 46–69 (2018). DOI 10.13154/tches.v2018.i2.46-69

Accepted manuscript

ACCEPTED MANUSCRIPT

25. Ibrahim, O., Hussain, A., Oligeri, G., Pietro, R.: Key is in the Air: Hacking Remote Keyless Entry Systems, p. 125–132. Springer International Publishing (2019). DOI 10.1007/978-3-030-16874-2_9
26. Immler, V.: Breaking Hitag 2 Revisited. In: Proceedings of the Second International Conference on Security, Privacy, and Applied Cryptography Engineering, SPACE'12, p. 126–143. Springer-Verlag, Berlin, Heidelberg (2012). DOI 10.1007/978-3-642-34416-9_9
27. Indesteege, S., Keller, N., Dunkelman, O., Biham, E., Preneel, B.: A Practical Attack on KeeLoq. In: N. Smart (ed.) Advances in Cryptology – EUROCRYPT 2008, pp. 1–18. Springer Berlin Heidelberg, Berlin, Heidelberg (2008). DOI 10.1007/978-3-540-78967-3_1
28. Joo, K., Choi, W., Lee, D.H.: Hold the Door! Fingerprinting Your Car Key to Prevent Keyless Entry Car Theft. In: Proceedings 2020 Network and Distributed System Security Symposium. Internet Society, San Diego, CA (2020). DOI 10.14722/ndss.2020.23107
29. Kamkar, S.: Drive It Like You Hacked It: New Attacks and Tools to Wirelessly Steal Cars (2015). URL https://defcon.org/html/defcon-23/dc-23-speakers.html#Kamkar
30. Kasper, M., Kasper, T., Moradi, A., Paar, C.: Breaking KeeLoq in a Flash: On Extracting Keys at Lightning Speed. In: B. Preneel (ed.) Progress in Cryptology – AFRICACRYPT 2009, pp. 403–420. Springer Berlin Heidelberg, Berlin, Heidelberg (2009)
31. Kendil, S.A.: Reverse engineering a car key fob signal (Part 1) (2024). URL https://0x44.cc/radio/2024/03/13/reversing-a-car-key-fob-signal.html
32. miek/inspectrum: Radio Signal Analyzer (2024). URL https://github.com/miek/inspectrum
33. Ossmann, M.: Rapid Radio Reversing. URL https://www.blackhat.com/docs/asia-16/materials/asia-16-Ossmann-Rapid-Radio-Reversing-wp.pdf
34. OZ9AEC, A.C.: Gqrx sdr - open source software defined radio by alexandru csete oz9aec (2024). URL https://gqrx.dk/
35. Pavel Zhovner, A.Z., Nadyrshin, R.: Our Response to the Canadian Government (2024). URL https://blog.flipper.net/response-to-canadian-government/
36. Pohl, J., Noack, A.: Universal Radio Hacker: A Suite for Analyzing and Attacking Stateful Wireless Protocols. In: 12th USENIX Workshop on Offensive Technologies (WOOT 18). USENIX Association, Baltimore, MD (2018). URL https://www.usenix.org/conference/woot18/presentation/pohl
37. Radu, A.I., Garcia, F.D.: Grey-box Analysis and Fuzzing of Automotive Electronic Components via Control-Flow Graph Extraction. In: Proceedings of the 4th ACM Computer Science in Cars Symposium, CSCS '20. Association for Computing Machinery, New York, NY, USA (2020). DOI 10.1145/3385958.3430480
38. UN Regulation No. 155 - Cyber security and cyber security management system. Tech. rep., UNECE (2021). URL https://unece.org/transport/documents/2021/03/standards/un-regulation-no-155-cyber-security-and-cyber-security
39. UN Regulation No. 156 - Software update and software update management system. Tech. rep., UNECE (2021). URL https://unece.org/transport/documents/2021/03/standards/un-regulation-no-156-software-update-and-software-update
40. Valdes-Dapena, P.: Some auto insurers are refusing to cover certain Hyundai and Kia models (2023). URL https://edition.cnn.com/2023/01/27/business/progressive-state-farm-hyundai-kia/index.html
41. Verdult, R., Garcia, F.D., Balasch, J.: Gone in 360 Seconds: Hijacking with Hitag2. In: 21st USENIX Security Symposium (USENIX Security 12), pp. 237–252. USENIX Association, Bellevue, WA (2012). URL https://www.usenix.org/conference/usenixsecurity12/technical-sessions/presentation/verdult
42. Verdult, R., Garcia, F.D., Ege, B.: Dismantling Megamos Crypto: Wirelessly Lockpicking a Vehicle Immobilizer. In: Supplement to the Proceedings of 22nd USENIX Security Symposium (Supplement to USENIX Security 15), pp. 703–718. USENIX Association, Washington, D.C. (2015). URL https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/verdult
43. Verstegen, A., Verdult, R., Bokslag, W.: Hitag 2 Hell – Brutally Optimizing Guess-and-Determine Attacks. In: 12th USENIX Workshop on Offensive Technologies (WOOT 18). USENIX Association, Baltimore, MD (2018). URL https://www.usenix.org/conference/woot18/presentation/verstegen
44. Driving Automotive Cybersecurity Forward. URL https://vicone.com
45. Pwn2Own Automotive. URL https://vicone.com/pwn2own-automotive
46. Wang, J., Lounis, K., Zulkernine, M.: CSKES: A Context-Based Secure Keyless Entry System. In: 2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC), vol. 1, p. 817–822 (2019). DOI 10.1109/COMPSAC.2019.00120
47. Wouters, L., Gierlichs, B., Preneel, B.: My other car is your car: compromising the Tesla Model X keyless entry system. IACR Transactions on Cryptographic Hardware and Embedded Systems **2021**(4), 149–172 (2021). DOI 10.46586/tches.v2021.i4.149-172
48. Wouters, L., Marin, E., Ashur, T., Gierlichs, B., Preneel, B.: Fast, furious and insecure: passive keyless entry and start systems in modern supercars. IACR Transactions on Cryptographic Hardware and Embedded Systems

Accepted manuscript

ACCEPTED MANUSCRIPT

2019(3), 66–85 (2019). DOI 10.13154/tches.v2019.i3.66-85

49. Yoshizawa, T., Preneel, B.: Post-Quantum Impacts on V2X Certificates – Already at The End of The Road. In: 2023 IEEE 97th Vehicular Technology Conference (VTC2023-Spring), pp. 1–6 (2023). DOI 10.1109/VTC2023-Spring57618.2023.10199793

Accepted manuscript